CS1200: Intro. to Algorithms and their Limitations	Prof. Anurag Anshu

Lecture 18: Resolution, Complexity Classes

Harvard SEAS - Spring 2026 *2026-04-08*

1 Announcements

- Anurag's extended OH today: 12:45-2:45
- SRE5 today!
- Pset 8 released.

Recommended Reading:

- Hesterberg–Vadhan 17.2-17.5, 18.1-18.2
- MacCormick 5.3-5.5, 10, 11

2 Recap: resolution theorem

Theorem 2.1 (Resolution Theorem)**.** *Let $\mathcal{C}$ be a set of clauses over n variables $x_0, \ldots, x_{n-1}$. Suppose that $\mathcal{C}$ is* closed under resolution*, meaning that for every $C, D \in \mathcal{C}$, we have $C \diamond D \in \mathcal{C} \cup \{1\}$. Then:*

1. *$(\ ) \in \mathcal{C}$ iff $\mathcal{C}$ is unsatisfiable.*
2. *If $(\ ) \notin \mathcal{C}$, then* `ExtractAssignment(`$\mathcal{C}$`)` *finds a satisfying assignment to $\mathcal{C}$ in time $O(n + k \cdot |\mathcal{C}|)$, where* `ExtractAssignment` *is an algorithm described in Section 3.*

Resolution algorithm: We start with the set of clauses $C_0, C_1, \cdots, C_{m-1}$ that appear in the CNF φ, simplify all the clauses in φ and then:

1. Resolve C_0 with each of $C_1, \ldots, C_{m-1}$, adding any new clauses obtained from the resolution $C_m, C_{m+1}, \ldots$. If the empty clause $(\ )$ is found, return `unsatisfiable`.

2. Resolve C_1 with each of $C_2, \ldots, C_{m-1}$ as well as with all of the resolvents obtained in Step 1, again adding any new clauses. If $(\ )$ is found, return `unsatisfiable`.

3. Resolve C_2 with each of $C_3, \ldots, C_{m-1}$ as well as with all of the resolvents obtained in Steps 1 and 2, again adding any new clauses. If ( ) is found, return `unsatisfiable`.

4. etc.

5. Run `ExtractAssignment` on the set of all clauses and return the satisfying assignment.

3 Assignment Extraction

We begin by describing the `ExtractAssignment` algorithm for finding a satisfying assignment to a set $\mathcal{C}$ of clauses that are closed under resolution and don't contain ( ). Specifically, we generate our satisfying assignment one variable v at a time in the following manner:

1. If $\mathcal{C}$ contains a singleton clause (v), then

2. If it contains a singleton clause $(\neg v)$ then

3. If it contains neither (v) nor $(\neg v)$, then

4. $\mathcal{C}$ cannot contain both (v) and $(\neg v)$, because

Once we have assigned a variable to a value, we set that variable's value in every clause and simplify.

Formally, the algorithm works as follows:

```
ExtractAssignment(𝒞):
Input  : A closed and simplified set 𝒞 of clauses over variables
         x_0, ..., x_{n-1} such that ( ) ∉ 𝒞
Output : An assignment α ∈ {0,1}^n that satisfies all of the clauses
         in 𝒞
foreach i = 0, ..., n-1 do
    if (x_i) ∈ 𝒞 then α_i = 1;
    else α_i = 0;
    𝒞 = 𝒞|_{x_i=α_i}, meaning that we set x_i = α_i and then simplify all clauses.
return α
```

Algorithm 3.2: Assignment extraction algorithm

Example 3.1. Consider applying Algorithm 3.2 to the set of clauses derived from the formula in Example from class:

$$(\neg x_0 \vee x_3), (x_0 \vee \neg x_3), (\neg x_1 \vee x_2), (\neg x_2 \vee x_1), (\neg x_3), (\neg x_0)$$

4 Proof Sketches of Runtime and Correctness

Proof Sketch of Resolution theorem. First, suppose $\mathcal{C}$ is such that $(\ ) \in \mathcal{C}$.

Conversely, if $(\ ) \notin \mathcal{C}$, then Item 2 says that `ExtractAssignment` will find a satisfying assignment, so $\mathcal{C}$ is satisfiable. Thus we turn to sketching the proof of Item 2. The key point is to show that after assigning a variable x_i as in `ExtractAssignment`, the set of clauses $\mathcal{C}|_{x_i=\alpha_i}$ satisfies the following:

$\square$

Now, we consider the runtime and correctness of the resolution algorithm. Let $\mathcal{C}_{fin}$ be the final set of clauses produced in Resolution Algorithm. Let k_{fin} be the maximum *width* (number of literals) among the clauses in $\mathcal{C}_{fin}$.

Runtime. Before analysing the runtime of the resolution algorithm, let's understand why the resolution algorithm terminates.

We can now give a finer estimate of the runtime of the resolution algorithm.

Correctness:

5 Efficient algorithm for 2SAT

Although the worst-case running time of resolution is exponential, the following is an important special case where it has polynomial runtime.

Input: A CNF formula φ on n variables in which each clause has width at most 2 (i.e. contains at most 2 literals) **Output:** An $\alpha \in \{0,1\}^n$ such that $\varphi(\alpha) = 1$, or $\perp$ if no satisfying assignment exists

Computational Problem 2SAT

Runtime of the resolution algorithm for 2SAT**:** observe that on a 2SAT instance φ, the Resolution algorithm will never create a clause of size larger than 2:

Corollary 5.1. 2SAT *can be solved in time* _____.

There is an algorithm known for 2SAT with runtime $O(n + m)$, where m is the number of clauses, by reduction to finding strongly connected components of directed graphs. (This algorithm is covered in CS1240.) Unfortunately, just like with coloring, once we switch from $k = 2$ to $k = 3$, the best known algorithms have exponential ($O(c^n)$) worst-case runtimes.

6 SAT Solvers

Enormous effort has gone into designing SAT Solvers that perform well on many real-world satisfiability instances, often but not always avoiding the worst-case exponential complexity. These methods are very related to Resolution. In some sense, they can be viewed as interleaving the `ExtractAssignment` algorithm and Resolution steps, in the hope of quickly finding either a satisfying assignment or a proof of unsatisfiability.

7 Computational complexity

In the remainder of the course, our focus is understanding why some computational problems seem resistant to satisfactory solutions—that is, what are the inherent *limits* on algorithms?

Computational complexity aims to classify problems according to the amount of resources (e.g. time) that they require to solve.

For example, we've seen problems whose fastest *known* algorithms run in:

- Linear time:
- Nearly linear time:
- Polynomial time:
- Exponential time:

Recall that the same problem can have many different algorithms each with a different runtime. An ultimate goal of Computational Complexity would be to identify the *fastest* possible runtime for a problem, so that once we have achieved it, we can stop trying to improve our algorithms further. As we will see, the field is not quite there yet, but we do have a very rich understanding of the complexity of problems.

To develop a robust and clean theory for classifying problems according to computational complexity, we make two choices:

A problem-independent size measure. To have a clean classification of problems, it is preferable to have one, single-parameter size measure that we apply across different problems. With Word-RAM as our main model of computation, it is natural to encode inputs x as arrays of N natural numbers, such that every array entry has $O(\log N)$ bits. The length N of the array x will be our size measure, and we will denote it by $|x|$.

<u>Examples:</u>

- Sorting On Finite Universe:
- Graph Problems:
- k-SAT:

Polynomial slackness in running time. We will only try to make coarse distinctions in running time, such as *polynomial time* vs. super-polynomial time. This choice is motivated in part by the Extended Church–Turing Thesis, which said the following (in our second formulation of it):

It is possible to make finer distinctions, like linear vs. nearly linear vs. quadratic, if we fix a model (like Word-RAM), and a newer area of research called *Fine-Grained Complexity* does this.

8 Complexity Classes

Following up on the above ideas, we define the following *complexity classes*.

Definition 8.1 (complexity classes)**.** For a function $T : \mathbb{N} \to \mathbb{R}^+$ with $T(n) \geq n+2$,

- $\mathsf{TIME}_{\mathsf{search}}(T(n))$ is

- $\mathsf{TIME}(T(n))$ is

- (Polynomial time)

$$\mathsf{P}_{\mathsf{search}} = \qquad\qquad \mathsf{P} = \qquad\qquad .$$

- (Exponential time)

$$\mathsf{EXP}_{\mathsf{search}} = \qquad\qquad \mathsf{EXP} = \qquad\qquad .$$

Note that $\mathsf{P}_{\mathsf{search}}$ and $\mathsf{EXP}_{\mathsf{search}}$ would be the same if we replace Word-RAM programs with algorithms described using any strongly Turing-equivalent model, like Turing Machines.

The algorithms that we have covered this semester demonstrate that a number of problems are in $\mathsf{P}_{\mathsf{search}}$. For some other problems, we only know that they are in $\mathsf{EXP}_{\mathsf{search}}$. Can we prove that they are not in $\mathsf{P}_{\mathsf{search}}$?